EE473 Final Project: Energy-Aware Edge Scheduling with Reinforcement Learning

Zilong Zeng & Meng Wu

1) Problem Setup

We model edge scheduling as a sequential decision problem with three actions: `low`, `medium`, and `high`. At each step, workload arrives from trace data. The chosen action sets service/energy behavior, then queue and battery states are updated.

Reward: `r_t = -(alpha_energy * energy_t + beta_latency * latency_t + gamma_miss * miss_t)`

Default weights: `(alpha, beta, gamma) = (1.0, 0.6, 2.0)`.

2) Dataset and Evaluation Protocol

- Source: Google cluster task-event trace (`120` shards)
- Preprocess: aggregate submit events, normalize workload, split train/test
- Episode length: `288`
- Main protocol: `20` train episodes + `6` non-overlap test episodes
- Methods: heuristic baselines, tabular Q-learning, linear approximation Q-learning
- Reporting: 5 seeds with mean/std

3) Environment

In [1]:
```
from pathlib import Path
import sys
import numpy as np
import matplotlib.pyplot as plt

ROOT = Path.cwd().resolve()
if not (ROOT / 'scripts').exists():
    ROOT = ROOT.parent
assert (ROOT / 'scripts').exists(), f'Expected repo root at {ROOT}'
print('ROOT =', ROOT)

SRC = ROOT / 'src'
if str(SRC) not in sys.path:
    sys.path.insert(0, str(SRC))

from config import DEFAULT_ENV_CONFIG
```

```
from data_loader import load_workload_trace, build_episodes
from env import EnergySchedulingEnv

TRACE_PATH = ROOT / 'data/processed/workload_trace_120.csv'
EPISODE_LENGTH = 288
STRIDE = 288

train_series = load_workload_trace(TRACE_PATH, split='train')
test_series  = load_workload_trace(TRACE_PATH, split='test')

train_episodes = build_episodes(train_series, EPISODE_LENGTH, stride=STRIDE)
test_episodes  = build_episodes(test_series,  EPISODE_LENGTH, stride=STRIDE)

print(f"Train episodes : {len(train_episodes)}  ×  {len(train_episodes[0])}
print(f"Test  episodes : {len(test_episodes)}  ×  {len(test_episodes[0])} st
print(f"Workload range : [{min(train_series):.3f}, {max(train_series):.3f}]"
```

```
ROOT = /Users/zilongzeng/Documents/EE473final project
Train episodes : 20  ×  288 steps
Test  episodes : 6  ×  288 steps
Workload range : [0.000, 1.000]
```

In [2]:

```
ep0 = np.array(train_episodes[0], dtype=float)
print('Episode 0 workload stats:')
print(f"  mean={ep0.mean():.4f}, std={ep0.std():.4f}, min={ep0.min():.4f}, m
```

```
Episode 0 workload stats:
  mean=0.0902, std=0.1681, min=0.0000, max=1.0000
```

In [3]:

```
# Run one short random-policy rollout
rng = np.random.default_rng(42)
env = EnergySchedulingEnv(train_episodes[0], config=DEFAULT_ENV_CONFIG)
obs, info = env.reset()
ret = 0.0
for _ in range(288):
    action = int(rng.integers(0, 3))
    obs, reward, done, info = env.step(action)
    ret += reward
    if done:
        break
print(f"Random policy episode return: {ret:.3f}")
print(f"Final queue={info['queue']:.3f}, final battery={info['battery']:.3f}
```

```
Random policy episode return: -151.711
Final queue=0.000, final battery=0.000
```

4) Baseline Evaluation

Evaluate fixed policies (`always_low` , `always_medium` , `always_high` , threshold) on held-out test episodes.

In [4]:

```
import pandas as pd
from baselines import (
    always_low_policy, always_medium_policy, always_high_policy,
    threshold_policy_factory, evaluate_policy,
```

```
)

baseline_policies = {
    'always_low': always_low_policy,
    'always_medium': always_medium_policy,
    'always_high': always_high_policy,
    'threshold(w=0.60)': threshold_policy_factory(workload_threshold=0.60),
    'threshold(w=0.80)': threshold_policy_factory(workload_threshold=0.80),
}

baseline_results = {}
rows = []
for name, policy in baseline_policies.items():
    m = evaluate_policy(test_episodes, policy, config=DEFAULT_ENV_CONFIG)
    baseline_results[name] = m
    rows.append({
        'Policy': name,
        'Avg Return': round(m['avg_episode_return'], 3),
        'Energy': round(m['avg_step_energy'], 4),
        'Latency': round(m['avg_step_latency'], 4),
        'Miss Rate': round(m['miss_rate'], 4),
    })

baseline_df = pd.DataFrame(rows).sort_values('Avg Return', ascending=False).
display(baseline_df)
```

	Policy	Avg Return	Energy	Latency	Miss Rate
0	always_low	-57.289	0.1800	0.0315	0.0
1	threshold(w=0.80)	-57.390	0.1951	0.0069	0.0
2	threshold(w=0.60)	-58.512	0.2022	0.0016	0.0
3	always_medium	-121.749	0.4200	0.0046	0.0
4	always_high	-244.800	0.8500	0.0000	0.0

5) RL Training (Single-Seed Demonstration)

We first run tabular and linear-approximation Q-learning once to verify training behavior. Final claims are based on the 5-seed artifacts shown later.

In [5]:
```
import time
from q_learning import QLearningConfig, train_tabular_q_learning

# ── Tabular Q-learning (300 epochs, single seed) ──────────────────────
tabular_cfg = QLearningConfig(
    num_epochs=300,
    alpha=0.15,
    gamma=0.98,
    epsilon_start=0.30,
    epsilon_end=0.02,
    epsilon_decay=0.995,
```

```
    eval_every=5,
    seed=42,
)

t0 = time.perf_counter()
tabular_result = train_tabular_q_learning(
    train_episodes=train_episodes,
    test_episodes=test_episodes,
    env_config=DEFAULT_ENV_CONFIG,
    algo_config=tabular_cfg,
)
tabular_time = time.perf_counter() - t0

print(f"Tabular Q-learning  —  training time: {tabular_time:.2f} s")
bm = tabular_result['best_test_metrics']
print(f"  Best checkpoint (epoch {int(tabular_result['best_epoch'])}):")
print(f"    avg_return = {bm['avg_episode_return']:.4f}")
print(f"    energy     = {bm['avg_step_energy']:.4f}")
print(f"    latency    = {bm['avg_step_latency']:.4f}")
print(f"    miss_rate  = {bm['miss_rate']:.4f}")
```

```
Tabular Q-learning  —  training time: 6.77 s
  Best checkpoint (epoch 115):
    avg_return = -56.2166
    energy     = 0.1878
    latency    = 0.0123
    miss_rate  = 0.0000
```

In [6]:

```
from approx_q import ApproxQLearningConfig, train_linear_approx_q_learning

# ── Linear Approximation Q-learning (400 epochs, single seed) ──────────────
approx_cfg = ApproxQLearningConfig(
    num_epochs=400,
    alpha=0.02,
    gamma=0.98,
    epsilon_start=0.30,
    epsilon_end=0.02,
    epsilon_decay=0.997,
    eval_every=5,
    seed=42,
)

t0 = time.perf_counter()
approx_result = train_linear_approx_q_learning(
    train_episodes=train_episodes,
    test_episodes=test_episodes,
    env_config=DEFAULT_ENV_CONFIG,
    algo_config=approx_cfg,
)
approx_time = time.perf_counter() - t0

print(f"Linear Approx Q-learning  —  training time: {approx_time:.2f} s")
print(f"  Feature vector dimension: {int(approx_result['feature_dim'])}")
bm = approx_result['best_test_metrics']
print(f"  Best checkpoint (epoch {int(approx_result['best_epoch'])}):")
print(f"    avg_return = {bm['avg_episode_return']:.4f}")
```

```
print(f"    energy     = {bm['avg_step_energy']:.4f}")
print(f"    latency    = {bm['avg_step_latency']:.4f}")
print(f"    miss_rate  = {bm['miss_rate']:.4f}")
```

```
Linear Approx Q-learning  —  training time: 36.48 s
  Feature vector dimension: 66
  Best checkpoint (epoch 260):
    avg_return = -55.9235
    energy     = 0.1898
    latency    = 0.0074
    miss_rate  = 0.0000
```

In [7]:

```
# Compact training summary (skip extra in-notebook learning-curve plot)
tab_hist = tabular_result['history']
apx_hist = approx_result['history']

best_baseline_return = max(v['avg_episode_return'] for v in baseline_results
best_tab = max(h['eval_test_avg_return'] for h in tab_hist)
best_apx = max(h['eval_test_avg_return'] for h in apx_hist)

print('Training summary (single-seed demo):')
print(f"  Best baseline return : {best_baseline_return:.3f}")
print(f"  Best tabular return  : {best_tab:.3f}")
print(f"  Best approx return   : {best_apx:.3f}")
print(f"  Training time (s)    : tabular={tabular_time:.2f}, approx={approx_
```

```
Training summary (single-seed demo):
  Best baseline return : -57.289
  Best tabular return  : -56.217
  Best approx return   : -55.923
  Training time (s)    : tabular=6.77, approx=36.48
```

6) Main Results (5-Seed Robust Protocol)

Below is the primary result set used for conclusions. Protocol: seeds `11, 22, 33, 44, 55`, `120` shards, non-overlap test episodes (`episode_length=288`, `stride=288`), and `20` train + `6` test episodes. All metrics are reported as mean ± std across seeds.

In [8]:

```
import json
from pathlib import Path
import pandas as pd
import matplotlib.image as mpimg

RESULTS = ROOT / 'results'
FIG_DIR = RESULTS / 'final_figures'

with open(RESULTS / 'phase3_multiseed_120.json') as f:
    ms = json.load(f)

def fmt(mean, std, digits=3):
    return f'{mean:.{digits}f} ± {std:.{digits}f}'

bm = ms['baseline']['best_test_metrics']
```

```
rows = [
    {
        'Method': 'Baseline (always_low)',
        'Return (mean±std)': fmt(bm['avg_episode_return'], 0.0),
        'Energy': fmt(bm['avg_step_energy'], 0.0, 4),
        'Latency': fmt(bm['avg_step_latency'], 0.0, 4),
        'Miss Rate': fmt(bm['miss_rate'], 0.0, 4),
        'Train(s)': '0.00 ± 0.00',
    }
]
for key, label in [('tabular', 'Tabular Q-learning'), ('approx', 'Linear App
    ag = ms[key]['aggregate_best_test_metrics']
    rows.append({
        'Method': label,
        'Return (mean±std)': fmt(ag['avg_episode_return']['mean'], ag['avg_e
        'Energy': fmt(ag['avg_step_energy']['mean'], ag['avg_step_energy']['
        'Latency': fmt(ag['avg_step_latency']['mean'], ag['avg_step_latency'
        'Miss Rate': fmt(ag['miss_rate']['mean'], ag['miss_rate']['std'], 4)
        'Train(s)': fmt(ag['training_wall_time_sec']['mean'], ag['training_w
    })

main_results_df = pd.DataFrame(rows)
display(main_results_df)

tab_ret = ms['tabular']['aggregate_best_test_metrics']['avg_episode_return']
apx_ret = ms['approx']['aggregate_best_test_metrics']['avg_episode_return']
tab_t = ms['tabular']['aggregate_best_test_metrics']['training_wall_time_sec
apx_t = ms['approx']['aggregate_best_test_metrics']['training_wall_time_sec'

tab_gain = tab_ret['mean'] - bm['avg_episode_return']
apx_gain = apx_ret['mean'] - bm['avg_episode_return']
print(f'Return gain vs baseline -> Tabular: {tab_gain:+.3f}, Approx: {apx_ga
print(f'Time ratio (Approx / Tabular): {apx_t["mean"] / tab_t["mean"]:.1f}x'
```

	Method	Return (mean±std)	Energy	Latency	Miss Rate	Train(s)
0	Baseline (always_low)	-57.289 ± 0.000	0.1800 ± 0.0000	0.0315 ± 0.0000	0.0000 ± 0.0000	0.00 ± 0.00
1	Tabular Q-learning	-56.200 ± 0.084	0.1907 ± 0.0011	0.0074 ± 0.0014	0.0000 ± 0.0000	6.34 ± 0.04
2	Linear Approx Q-learning	-56.016 ± 0.117	0.1897 ± 0.0005	0.0081 ± 0.0010	0.0000 ± 0.0000	34.39 ± 0.17

```
Return gain vs baseline -> Tabular: +1.088, Approx: +1.272
Time ratio (Approx / Tabular): 5.4x
```

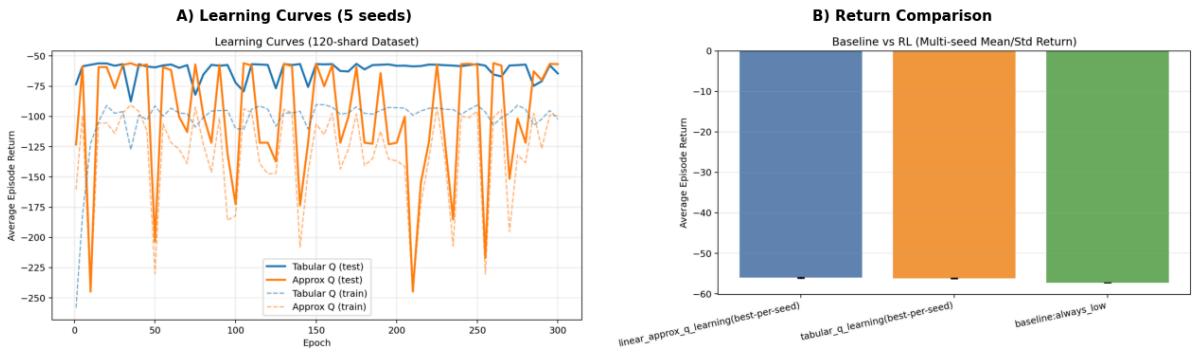
Main Figures (From Saved 5-Seed Artifacts)

How to read these figures:

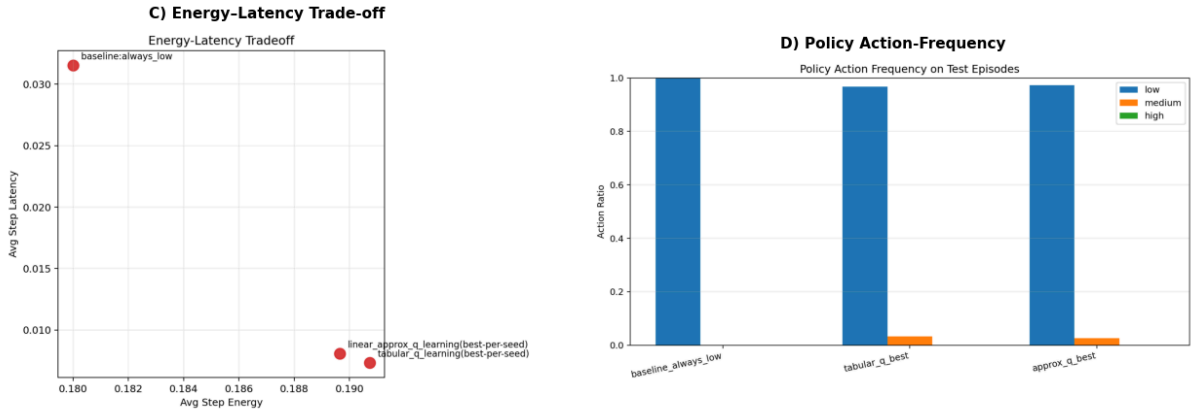
- Learning-curve panel: checks convergence trend and seed-to-seed variance.

- Return-comparison panel: checks final performance gaps between baseline and RL methods.
- Trade-off panel: compares energy and latency together (not just color differences).
- Action-frequency panel: shows whether policies actually choose different control actions.

In [9]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
for ax, fname, title in [
    (axes[0], 'learning_curves_tabular_vs_approx.png', 'A) Learning Curves (
    (axes[1], 'baseline_vs_rl_return.png',             'B) Return Comparisor
]:
    ax.imshow(mpimg.imread(FIG_DIR / fname))
    ax.axis('off')
    ax.set_title(title, fontsize=11, fontweight='bold')
plt.tight_layout()
plt.show()
```



In [10]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
for ax, fname, title in [
    (axes[0], 'energy_latency_tradeoff.png', 'C) Energy\u2013Latency Trade-c
    (axes[1], 'policy_action_frequency.png', 'D) Policy Action-Frequency'),
]:
    ax.imshow(mpimg.imread(FIG_DIR / fname))
    ax.axis('off')
    ax.set_title(title, fontsize=11, fontweight='bold')
plt.tight_layout()
plt.show()
```



7) Policy Snapshot (Current Run)

This section evaluates the two models trained in this notebook run (single seed) on the same test split. It serves as a sanity check that the live run and saved artifacts are directionally consistent.

In [11]:
```
from q_learning import evaluate_q_policy
from approx_q import LinearQApproximator, evaluate_linear_q_policy

# Models from this notebook run
q_table_run = tabular_result['best_q_table']
weights_run = approx_result['best_weights']
approximator_run = LinearQApproximator(DEFAULT_ENV_CONFIG)

tab_run = evaluate_q_policy(q_table_run, test_episodes, env_config=DEFAULT_E
apx_run = evaluate_linear_q_policy(weights_run, approximator_run, test_episc

print('Current-run model evaluation on test episodes')
print('-' * 72)
print(f"{'Method':<24} {'Avg Return':>10} {'Energy':>8} {'Latency':>8} {'Mis
print('-' * 72)
for name, m in [('Tabular Q (run best)', tab_run), ('Approx Q (run best)', a
    print(f"{name:<24} {m['avg_episode_return']:>10.3f} {m['avg_step_energy'
```

```
Current-run model evaluation on test episodes
------------------------------------------------------------------------
Method                    Avg Return   Energy  Latency     Miss
------------------------------------------------------------------------
Tabular Q (run best)        -56.217    0.188    0.012   0.0000
Approx Q (run best)         -55.923    0.190    0.007   0.0000
```

Feature Design (Linear Approximation)

`Q(s,a) = w_a^T phi(s)` with action-wise blocks.

Features include:

- bias and continuous terms (`workload`, normalized `queue`, `battery_ratio`)
- one-hot bins for workload/queue/battery
- interaction terms

Total feature dimension: `66`.

In [12]:
```
# Greedy-action distribution from the current-run tabular model
action_names = DEFAULT_ENV_CONFIG.action_names
n_w = len(DEFAULT_ENV_CONFIG.workload_bins) - 1
n_q = len(DEFAULT_ENV_CONFIG.queue_bins) - 1
counts = {a: 0 for a in action_names}
for w in range(n_w):
    for q in range(n_q):
        a = int(np.argmax(q_table_run[w, q, 4]))  # battery_bin = 4
```

```
        counts[action_names[a]] += 1

total = sum(counts.values())
print('Greedy action distribution (workload_bin x queue_bin, battery_bin=4):
for k, v in counts.items():
    print(f"  {k:<6}: {v}/{total} ({100*v/total:.1f}%)")
```

```
Greedy action distribution (workload_bin x queue_bin, battery_bin=4):
  low   : 8/25 (32.0%)
  medium: 7/25 (28.0%)
  high  : 10/25 (40.0%)
```

8) Results (Current Run)

The table and figure below summarize the current execution, mainly to confirm reproducibility.

In [13]:

```
# Build a compact current-run comparison table
best_baseline_name = max(baseline_results, key=lambda k: baseline_results[k]
best_baseline = baseline_results[best_baseline_name]

results_run = [
    {
        'Method': f'Baseline ({best_baseline_name})',
        'Return': round(best_baseline['avg_episode_return'], 3),
        'Energy': round(best_baseline['avg_step_energy'], 3),
        'Latency': round(best_baseline['avg_step_latency'], 3),
        'Miss': round(best_baseline['miss_rate'], 4),
        'Train(s)': 0.0,
    },
    {
        'Method': 'Tabular Q (run best)',
        'Return': round(tab_run['avg_episode_return'], 3),
        'Energy': round(tab_run['avg_step_energy'], 3),
        'Latency': round(tab_run['avg_step_latency'], 3),
        'Miss': round(tab_run['miss_rate'], 4),
        'Train(s)': round(tabular_time, 2),
    },
    {
        'Method': 'Approx Q (run best)',
        'Return': round(apx_run['avg_episode_return'], 3),
        'Energy': round(apx_run['avg_step_energy'], 3),
        'Latency': round(apx_run['avg_step_latency'], 3),
        'Miss': round(apx_run['miss_rate'], 4),
        'Train(s)': round(approx_time, 2),
    },
]

display(pd.DataFrame(results_run))

ret_gap_tab = tab_run['avg_episode_return'] - best_baseline['avg_episode_ret
ret_gap_apx = apx_run['avg_episode_return'] - best_baseline['avg_episode_ret
print('Return gains vs best baseline:')
```

```
print(f'  Tabular: {ret_gap_tab:+.3f}')
print(f'  Approx : {ret_gap_apx:+.3f}')
```

	Method	Return	Energy	Latency	Miss	Train(s)
0	Baseline (always_low)	-57.289	0.180	0.032	0.0	0.00
1	Tabular Q (run best)	-56.217	0.188	0.012	0.0	6.77
2	Approx Q (run best)	-55.923	0.190	0.007	0.0	36.48

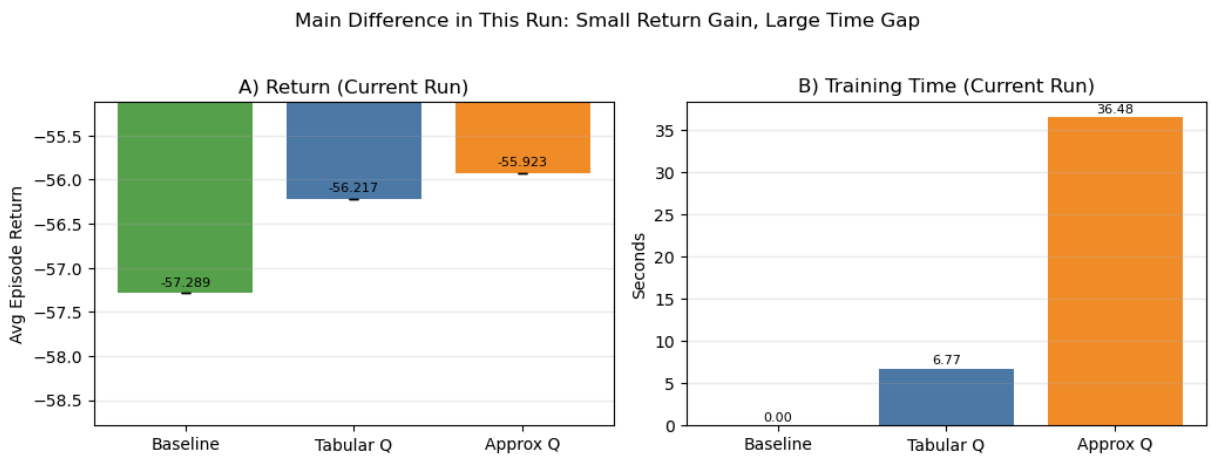
```
Return gains vs best baseline:
  Tabular: +1.072
  Approx : +1.365
```

Core Figure (Current Run)

Left: return comparison from this run. Right: training-time comparison. Together they show the key trade-off: small return gain versus large runtime gap.

In [14]:

```
import matplotlib.pyplot as plt

labels = ['Baseline', 'Tabular Q', 'Approx Q']
returns = [best_baseline['avg_episode_return'], tab_run['avg_episode_return'
# Use std=0 for this single run view; bars remain visually comparable.
ret_std = [0.0, 0.0, 0.0]
times = [0.0, tabular_time, approx_time]
colors = ['#59a14f', '#4e79a7', '#f28e2b']

fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.8))

ax = axes[0]
bars = ax.bar(labels, returns, yerr=ret_std, capsize=3, color=colors)
ax.set_title('A) Return (Current Run)')
ax.set_ylabel('Avg Episode Return')
ax.grid(axis='y', alpha=0.25)
ax.set_ylim(min(returns) - 1.5, max(returns) + 0.8)
for b, r in zip(bars, returns):
    ax.text(b.get_x() + b.get_width()/2, r + 0.05, f'{r:.3f}', ha='center',

ax2 = axes[1]
bars2 = ax2.bar(labels, times, color=colors)
ax2.set_title('B) Training Time (Current Run)')
ax2.set_ylabel('Seconds')
ax2.grid(axis='y', alpha=0.25)
for b, t in zip(bars2, times):
    ax2.text(b.get_x() + b.get_width()/2, t + 0.2, f'{t:.2f}', ha='center',

fig.suptitle('Main Difference in This Run: Small Return Gain, Large Time Gap
plt.tight_layout()
plt.show()

print(f"Time ratio (Approx/Tabular): {approx_time/tabular_time:.2f}x")
```




```
Time ratio (Approx/Tabular): 5.39x
```

9) Sensitivity Checks (Current Run)

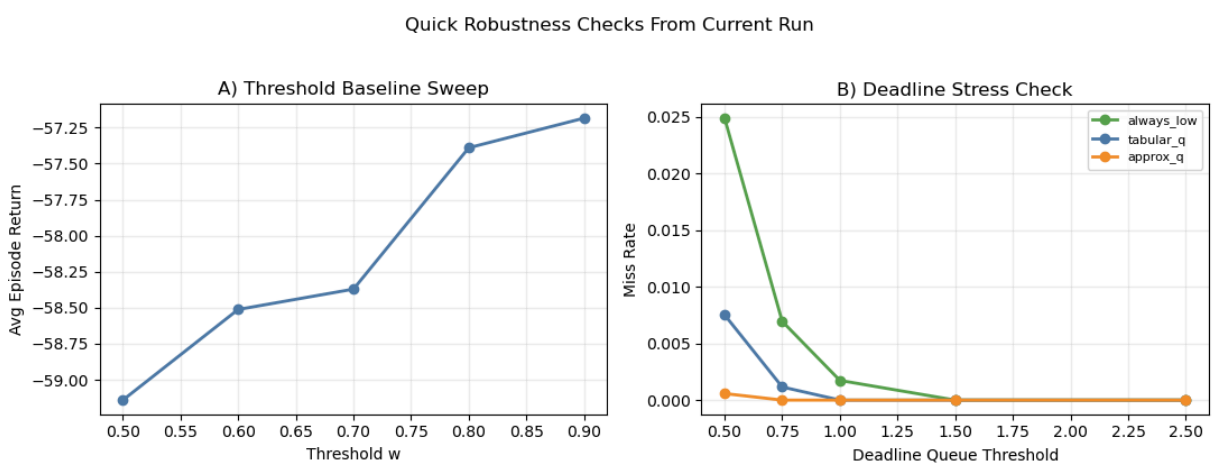
Two lightweight checks are included:

- threshold-policy sweep (baseline sensitivity),
- deadline-stress check (whether miss-rate differences become visible).

In [15]:

```
import matplotlib.pyplot as plt
from dataclasses import replace
from baselines import threshold_policy_factory

# A) Threshold sweep on the same test split
thresholds = [0.50, 0.60, 0.70, 0.80, 0.90]
th_returns = []
for w in thresholds:
    m = evaluate_policy(test_episodes, threshold_policy_factory(workload_thr
    th_returns.append(m['avg_episode_return'])

# B) Deadline stress: compare miss-rate for three policies
stress_deadlines = [2.5, 1.5, 1.0, 0.75, 0.5]

# Policy wrappers from current-run models
def tab_policy(obs, info):
    return int(np.argmax(q_table_run[obs]))

def apx_policy(obs, info):
    return approximator_run.greedy_action(weights_run, obs, info)

miss_tab = []
miss_apx = []
miss_low = []
for d in stress_deadlines:
    cfg = replace(DEFAULT_ENV_CONFIG, deadline_queue_threshold=d)
    miss_low.append(evaluate_policy(test_episodes, always_low_policy, confic
    miss_tab.append(evaluate_policy(test_episodes, tab_policy, config=cfg)['
    miss_apx.append(evaluate_policy(test_episodes, apx_policy, config=cfg)['

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
```

```
ax = axes[0]
ax.plot(thresholds, th_returns, marker='o', linewidth=2, color='#4e79a7')
ax.set_xlabel('Threshold w')
ax.set_ylabel('Avg Episode Return')
ax.set_title('A) Threshold Baseline Sweep')
ax.grid(alpha=0.25)

ax2 = axes[1]
ax2.plot(stress_deadlines, miss_low, marker='o', linewidth=2, label='always_
ax2.plot(stress_deadlines, miss_tab, marker='o', linewidth=2, label='tabular
ax2.plot(stress_deadlines, miss_apx, marker='o', linewidth=2, label='approx_
ax2.set_xlabel('Deadline Queue Threshold')
ax2.set_ylabel('Miss Rate')
ax2.set_title('B) Deadline Stress Check')
ax2.grid(alpha=0.25)
ax2.legend(fontsize=8)

fig.suptitle('Quick Robustness Checks From Current Run', y=1.03)
plt.tight_layout()
plt.show()

best_idx = int(np.argmax(th_returns))
print('Readable checkpoints:')
print(f"  Best threshold in sweep: w={thresholds[best_idx]:.2f}, return={th_
print(f"  Miss-rate @ deadline 0.5 -> always_low={miss_low[-1]:.4f}, tabular
```




```
Readable checkpoints:
  Best threshold in sweep: w=0.90, return=-57.184
  Miss-rate @ deadline 0.5 -> always_low=0.0249, tabular=0.0075, approx=0.00
06
```

10) Ablation Studies

Three ablation experiments test whether the main result is robust. All ablations use 5 seeds; linear approximation Q-learning is used as the representative RL agent.

10a) Reward-Weight Sensitivity

We vary `alpha` (energy) or `beta` (latency) while fixing the other terms. To avoid scale effects, we compare RL vs `always_low` within each reward setting.

In [16]:
```
with open(RESULTS / 'reward_sensitivity_context_120.json') as f:
    rs = json.load(f)

reward_rows = []
for r in rs['rows']:
    reward_rows.append({
        'Weights (alpha,beta,gamma)': f'({r["alpha_energy"]:.2f}, {r["beta_l
        'Method Return (mean±std)': f'{r["avg_episode_return_mean"]:.3f} ± {
        'AlwaysLow Return': round(r['always_low_return'], 3),
        'Gap vs AlwaysLow': round(r['gap_vs_always_low'], 3),
        'Relative Gain (%)': round(r['relative_gain_pct'], 2),
    })

display(pd.DataFrame(reward_rows))
print('Takeaway: RL stays above always_low under every tested reward-weight
```

	Weights (alpha,beta,gamma)	Method Return (mean±std)	AlwaysLow Return	Gap vs AlwaysLow	Relative Gain (%)
0	(1.00, 0.80, 2.00)	-56.463 ± 0.110	-59.105	2.642	4.47
1	(0.80, 0.60, 2.00)	-45.098 ± 0.050	-46.921	1.822	3.88
2	(1.00, 0.60, 2.00)	-56.027 ± 0.111	-57.289	1.262	2.20
3	(1.20, 0.60, 2.00)	-66.996 ± 0.126	-67.657	0.661	0.98
4	(1.00, 0.40, 2.00)	-55.397 ± 0.114	-55.472	0.075	0.14

```
Takeaway: RL stays above always_low under every tested reward-weight settin
g.
```

10b) Hyperparameter Sensitivity

We sweep learning rate (`alpha`), epsilon decay, and discount factor (`gamma`). Each configuration is evaluated with 5 seeds, and we aggregate the best test return per seed.

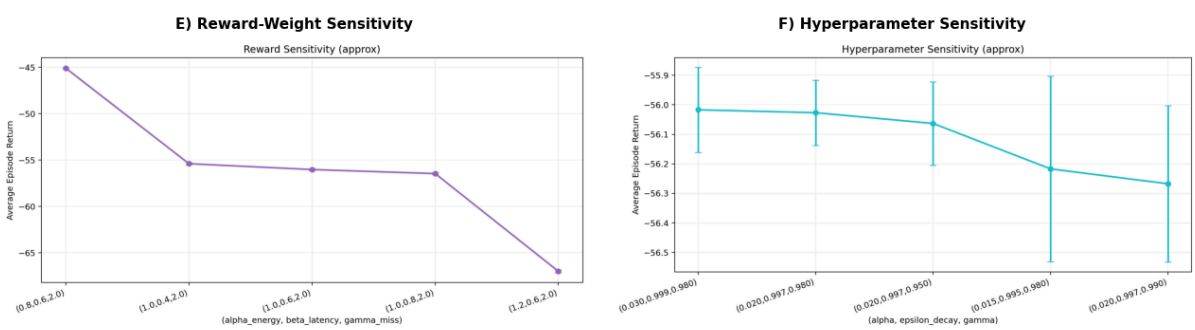
In [17]:
```
from collections import defaultdict

with open(RESULTS / 'hyperparam_sensitivity_120.json') as f:
    hp = json.load(f)

agg = defaultdict(list)
for rec in hp['records']:
    key = (rec['alpha'], rec['epsilon_decay'], rec['gamma'])
    for run in rec['runs']:
        agg[key].append(run['best_test_metrics']['avg_episode_return'])

rows = []
for (alpha, eps_decay, gamma), vals in agg.items():
    rows.append({
        'alpha': alpha,
```

```
        'epsilon_decay': eps_decay,
        'gamma': gamma,
        'Avg Return': round(float(np.mean(vals)), 3),
        'Std': round(float(np.std(vals)), 3),
        'Default Config': (alpha == 0.02 and eps_decay == 0.997 and gamma ==
    })

hp_df = pd.DataFrame(rows).sort_values('Avg Return', ascending=False).reset_
display(hp_df)
print('Takeaway: top configurations are close; sensitivity is limited in thi
```

	alpha	epsilon_decay	gamma	Avg Return	Std	Default Config
0	0.030	0.999	0.98	-56.017	0.144	False
1	0.020	0.997	0.98	-56.027	0.111	True
2	0.020	0.997	0.95	-56.063	0.141	False
3	0.015	0.995	0.98	-56.217	0.314	False
4	0.020	0.997	0.99	-56.267	0.265	False

```
Takeaway: top configurations are close; sensitivity is limited in this rang
e.
```

In [18]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
for ax, fname, title in [
    (axes[0], 'reward_sensitivity_return.png',    'E) Reward-Weight Sensitiv
    (axes[1], 'hyperparam_sensitivity_return.png','F) Hyperparameter Sensiti
]:
    ax.imshow(mpimg.imread(FIG_DIR / fname))
    ax.axis('off')
    ax.set_title(title, fontsize=11, fontweight='bold')
plt.tight_layout()
plt.show()
```



10c) Train/Test Slicing Generalization

We compare two slicing strategies to test whether the learned policy generalizes beyond one specific split. The same 5 seeds are used for a fair comparison.

In [19]:
```
with open(RESULTS / 'generalization_check_120.json') as f:
    gc = json.load(f)
```

```
rows = []
for rec in gc['records']:
    rets = [r['best_test_metrics']['avg_episode_return'] for r in rec['runs'
    engs = [r['best_test_metrics']['avg_step_energy'] for r in rec['runs']]
    lats = [r['best_test_metrics']['avg_step_latency'] for r in rec['runs']]
    miss = [r['best_test_metrics']['miss_rate'] for r in rec['runs']]
    rows.append({
        'Setting': rec['setting']['name'],
        'Train/Test': f"{rec['num_train_episodes']}/{rec['num_test_episodes'
        'Return (mean±std)': f"{np.mean(rets):.3f} ± {np.std(rets):.3f}",
        'Energy': round(float(np.mean(engs)), 4),
        'Latency': round(float(np.mean(lats)), 4),
        'Miss Rate': round(float(np.mean(miss)), 4),
    })

display(pd.DataFrame(rows))
print('Takeaway: performance is similar across slicing strategies, suggestir
```

	Setting	Train/Test	Return (mean±std)	Energy	Latency	Miss Rate
0	non_overlap	20/6	-45.348 ± 0.406	0.1924	0.0059	0.0
1	overlap_test	20/10	-45.709 ± 0.400	0.1933	0.0068	0.0

```
Takeaway: performance is similar across slicing strategies, suggesting decen
t generalization.
```

11) Answers to Required Questions

Course Question	Finding
Algorithm training time?	Tabular Q: ~6.3 s; Approx Q: ~34.4 s. Tabular is about **5.4× faster**.
How close to expected results?	Both RL methods beat the best heuristic (`always_low`) by about +1.1 to +1.3 return units, and this trend is stable across seeds.
How were parameters adjusted?	We tuned learning rate, epsilon decay, and discount factor via grid search; the default setting stayed near the top tier.
Which parameters mattered most?	Reward weight `beta` strongly affects absolute return scale; state discretization controls the coverage-vs-noise trade-off for tabular updates.
Main challenges?	Choosing discretization granularity, balancing multi-term reward weights, and exposing miss-rate differences under realistic deadline thresholds.
Future improvements?	Multi-device coordination, stricter SLA constraints, larger action spaces, and actor-critic / constrained RL methods.

Quantitative summary (5 seeds, 120 shards, non-overlap test):

- Best baseline `always_low` : `-57.289 ± 0.000`
- Tabular Q-learning: `-56.200 ± 0.084` (gain `+1.089` , `6.3 s`)

- Linear Approx Q-learning: `−56.016 ± 0.117` (gain `+1.273`, `34.4 s`)
- RL advantage is also consistent across all reward-weight settings and both slicing strategies.

12) Challenges and Future Work

Main implementation challenges

- balancing state detail and tabular coverage,
- calibrating reward weights without over-optimizing one metric,
- revealing miss-rate differences under realistic constraints.

Next technical steps

- evaluate on more workload slices and stricter constraints,
- test larger action/state spaces with stronger function approximators,
- extend from single-device control to coordinated multi-device scheduling.

In [20]:
```
print('Challenge Evidence Snapshot')
print('-' * 64)
print(f"Return gain vs best baseline (Tabular): {ret_gap_tab:+.3f}")
print(f"Return gain vs best baseline (Approx) : {ret_gap_apx:+.3f}")
print(f"Training-time ratio (Approx/Tabular) : {approx_time/tabular_time:.2f
print(f"Miss-rate @ strict deadline 0.5      : low={miss_low[-1]:.4f}, tab={
print('-' * 64)
```

```
Challenge Evidence Snapshot
----------------------------------------------------------------
Return gain vs best baseline (Tabular): +1.072
Return gain vs best baseline (Approx) : +1.365
Training-time ratio (Approx/Tabular) : 5.39x
Miss-rate @ strict deadline 0.5      : low=0.0249, tab=0.0075, approx=0.0006
----------------------------------------------------------------
```

13) Conclusion

This project models energy-aware edge scheduling as an RL problem and compares two course-aligned RL methods with heuristic baselines on Google cluster traces.

Main findings (5 seeds, 120 shards)

Method	Return (mean ± std)	Train Time
Baseline (always_low)	`−57.289 ± 0.000`	—
Tabular Q-learning	`−56.200 ± 0.084`	~6.3 s
Linear Approx Q-learning	`−56.016 ± 0.117`	~34.4 s

- Both RL methods consistently outperform the best heuristic.
- Approx Q reaches slightly better return, while Tabular Q is much faster to train.
- Ablation results show this conclusion is stable under reward and hyperparameter changes.

Practical takeaway: use Tabular Q for quick iteration, and use Approx Q when you prioritize final return quality.